

Auditing Static Analysis of Agent Workflows: Extraction Fidelity and Policy Applicability

Anonymous submission

Abstract

Static analysis of agent workflows depends on two choices that are often left implicit: whether the extracted graph represents the source, and whether a policy applies to the workflow being checked. We audit both choices on 922 extracted file-level graphs from public GitHub across LangGraph, CrewAI, AutoGen, and Google ADK. On a quota-selected 119-graph sample, an LLM-assisted source audit labels 184 of 186 findings non-actionable; these are exploratory labels pending independent human validation. Attribution is *check-family dependent*: a graph-substitution survival test attributes 68 of 72 non-actionable structural findings to extraction, whereas it attributes 94 of 112 non-actionable human-gate findings to an inapplicable approval policy. Holding checks fixed and substituting source-reconstructed graphs removes 62 of 184 findings. A declaration-sensitive human-gate rule moves blanket firings from 113 to 0, but only because mined graphs expose no sensitive bindings. The reverse audit exposes the consequence: all 12 source-audited human-gate violations escape the mined pipeline, and a source-reconstructed graph recovers only one; nine effects occur inside node bodies that the graph vocabulary does not represent. Moreover, every side-effecting workflow cleared by both graph checks was cleared by a *placebo* gate—a node typed `HUMAN` whose body cannot block. We therefore separate extraction fidelity, policy applicability, and abstraction coverage, and make `safe` conditional on an authored event-complete abstraction; otherwise the analyzer returns `inconclusive`. The result is a methodological requirement for agent-workflow analysis: validate both the recovered model and the applicability of every policy.

1 Introduction

Large language models are increasingly deployed as autonomous agents that call external tools and APIs (Schick et al. 2023; Yao et al. 2023), and orchestration frameworks structure this behavior as explicit *workflow graphs*: LangGraph (LangChain 2025), CrewAI (CrewAI 2025), AutoGen (Wu et al. 2024), and Google ADK (Google 2025) represent computation steps as nodes and allowable transitions as edges. Safety enforcement remains almost entirely a *runtime* concern: NeMo Guardrails (NVIDIA 2023) and

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

LlamaGuard (Inan et al. 2023) filter individual calls at execution time, detecting a violation only when the offending path is exercised.

Some defects, however, are topological. Consider an email-triage agent whose developer adds a `draft_response` node but forgets to connect it to `send`: normal-priority emails silently halt at a dead end. A runtime guardrail cannot catch this unless that path runs, but a static analysis of the graph flags it and emits a witness trace. Classical model checkers (Clarke, Grumberg, and Peled 1999; Holzmann 1997; Cimatti et al. 2002) verify such properties, but require hand-translation into Promela or SMV. For conventional static analyzers, adoption is decided not by detection power but by the rate of actionable findings (Bessey et al. 2010; Johnson et al. 2013; Sadowski et al. 2018); the analogous question for agent workflows—how often these graphs contain the defects static checks target—has received far less measurement.

Thesis. Answering it requires a prior question: *can we trust the graph?* A static check is only as credible as the behavioral model an extractor recovers from source, and on public agent code that extraction is lossy—it manufactures false flags and hides true defects at once. A second question is equally necessary: *does the policy govern this workflow?* Our audit shows that the answer is check-family dependent. A graph-substitution survival test attributes 68 of 72 non-actionable structural findings to extraction, while it attributes 94 of 112 non-actionable human-gate findings to an inapplicable blanket approval policy. These counts should not be pooled into a universal error rate: the aggregate is determined by the chosen mix of checks, frameworks, and files. More importantly, the reverse audit finds a different boundary. All 12 source-audited human-gate violations escape the mined pipeline; nine remain invisible even in a source-reconstructed graph because the regulated effect lives inside a node body. Static verification of agent workflows is therefore a measurement problem over three distinct objects: extraction fidelity, policy applicability, and abstraction coverage.

Study. We use it as a measurement instrument on 922 extracted file-level graphs mined from public GitHub across the four frameworks—a search-ranked convenience corpus, not a probability sample—and validate a quota-selected 119-graph sample against *LLM-reconstructed reference graphs* from an

independent, adversarially cross-checked LLM pass. These are *LLM-assisted exploratory labels*, not human-validated ground truth; a two-human validation protocol and locked samples are provided, and we bound our current claims accordingly.

Three audit quantities, not one prevalence. A pooled rate conflates distinct quantities, so we report separately: flag-level precision; observed structural defects, a topological judgment; and observed human-gate policy violations, a normative judgment under one policy fixed before case labeling (Table 3). Because neither the GitHub corpus nor the 119-file validation set is a probability sample, these are descriptive audit proportions, not population-prevalence estimates.

Contributions.

- **A cross-framework extraction-fidelity audit** with matched, provenance-disambiguated instruments and a corrected-extractor ablation. It localizes extraction error to structural checks instead of treating all findings as one population.
- **Symmetric audits of findings and misses.** Holding checks fixed while swapping mined for reconstructed graphs separates extraction from check-side effects; applying the same test to source-audited violations shows when the graph vocabulary itself omits the regulated behavior.
- **A separation of policy applicability from graph fidelity.** A policy fixed before case labeling exposes violations outside both original flag universes and *placebo* human gates that make graph-only evaluation overstate oversight.
- **A conservative operational consequence:** results are *safe* only relative to a caller-asserted, event-complete authored abstraction; lossy mined graphs return *inconclusive*. A small path-sensitive experiment demonstrates the resulting monitor-selection mechanism (Section 4).

2 Audit Instrument and Conditional Guarantees

Unified graph model. The analyzer represents a workflow as a directed graph $G = (V, E, v_0, V_f)$ with entry v_0 and exit set V_f . Each node carries a kind $\kappa_V(v)$ over ENTRY, EXIT, TOOL, LLM, ROUTER, HUMAN, PASSTHROUGH, SUBGRAPH and, for tool nodes, bound tool names; each edge is DIRECT, CONDITIONAL, PARALLEL, or LOOP. Every node and edge also carries extraction *provenance*—an origin (runtime object, explicit AST, inferred, synthesized, unknown), a confidence (exact, may, heuristic), and a source span—so structure read from a declaration is distinguishable from structure the extractor guessed, and witness traces inherit the weakest confidence they traverse. Per-framework extractors bridge four representations into this model by either of two paths: a runtime extractor reading the compiled graph object, or a static AST fallback for large-scale mining, where installing every repository’s dependencies is infeasible.

Event attributes. Nodes may additionally carry non-exclusive effects (read, write, execute, communicate, financial, delete) and capabilities. The mined extractors do not populate these fields from node bodies; corpus construction assigns them from declared tool names. This limitation is central to the audit rather than hidden by the representation.

Structural checks. Six checks run over the topology, each in $O(|V| + |E|)$ and each emitting a *witness trace* (Clarke, Grumberg, and Peled 1999) on failure: *exit reachability* ($V_f \subseteq \text{Reach}(v_0)$), *reverse reachability* (reachable nodes with no path to an exit), *dead-end detection* (non-exit nodes with no successor), *router shape* (router outgoing edges must be conditional), *human presence* (no HUMAN node—a deliberately blunt policy check we evaluate against a risk-aware variant), and *tool declaration* (tool nodes with an empty tool set). A seventh, path-sensitive check—*human-gate coverage*—flags a workflow only if a *sensitive* tool is reachable from v_0 without passing a HUMAN node (Section 3).

Temporal policies with LTL_f semantics. It also supports a policy DSL over finite traces (LTL_f (De Giacomo and Vardi 2013)): an execution is a finite event sequence, and each of seven DSL forms denotes an LTL_f formula—*forbidden* ($G \neg a$), *response* ($G(a \rightarrow F b)$), *strong until*, *bounded response* ($G(a \rightarrow F_{\leq k} b)$), *response chain*, *conjunction*, *disjunction* (grammar in the supplement). Formulas compile compositionally to a DFA by formula progression (2–3 states for base patterns, $k+2$ for bounded response), with two disjoint violation mechanisms: *violation states* are exactly the doomed states, so a bad prefix is flagged at the earliest event making every extension violating; and a pending *F-obligation* violates at termination. The compiler is differentially tested against an independent reference evaluator, exhaustively to length six for two-atom formulas. Runtime monitors evaluate events in $O(1)$; obligations open when a run aborts are reported *inconclusive* (Bauer, Leucker, and Schallhart 2011).

Static temporal verification. Policies are checked statically via a graph $\times$ DFA product: BFS from (v_0, q_0) advances the DFA at each node and follows graph edges, modeling executions as finite *maximal* paths. Because static extraction cannot prove which of a tool node’s declared tools will be invoked, or whether any will be, the product is *conservative*: at each tool node it closes the DFA state over every finite invocation sequence drawn from the declared tools, including the empty one—a false alarm is acceptable, a false proof is not. The checker returns *may-violate* (some resolution reaches a bad prefix or a pending obligation at termination), *safe* (none does), or *inconclusive* (a reachable product cycle postpones an obligation forever, or no termination point is reachable; an engineering-scope choice, not undecidability). Explored states are bounded by $|V| \cdot |Q|$.

Soundness requires an event-complete abstraction. Soundness is a claim about *event* traces, not graph paths. Let $\text{Traces}_{\text{ev}}(P)$ be the concrete finite terminating event traces a workflow P can emit, and let λ label each event, inducing a graph event-language $\mathcal{L}(G, \lambda)$. If $\text{Traces}_{\text{ev}}(P) \subseteq \mathcal{L}(G, \lambda)$

(*event-trace conservatism*) and every trace in $\mathcal{L}(G, \lambda)$ is accepted by the policy DFA, then every concrete finite terminating execution of P satisfies the policy—the guarantee we exploit for monitor pruning (Section 4); the proof is in the supplement. Graph-*path* containment alone does *not* suffice: a node may issue tool calls the graph never records. Because provenance cannot establish that node bodies emit no unmodeled events, the checker returns *safe* only when the caller explicitly asserts event-trace conservatism for an authored abstraction. Without that assertion, a would-be *safe* result is downgraded to *inconclusive* with reason `uncertified_extraction`; `may_violate` findings remain visible.

What the gate does and does not certify. Exact provenance describes *how* an element was obtained, not whether the abstraction covers all runtime events. An edge can be read exactly while a tool call inside its node body is absent. We therefore use provenance to qualify witnesses, never to discharge event-trace conservatism. The product is sound *relative to an authored abstraction* whose event coverage the caller asserts; on every mined graph in this study, the gate withholds *safe*. Independent event-coverage analysis remains future work.

3 Audit of Public GitHub Workflow Files

To audit extractor fidelity and finding actionability on independently authored code, we mined public GitHub across all four frameworks. The frame is *public GitHub files matching framework-specific code searches*, not production deployments; the unit is one source file mapped to one *extracted file-level graph*, so a fully executable-workflow-root unit is future work. All reported proportions describe this audit and its instrument; they are not population-prevalence estimates.

Mining and corpus census. We mined GitHub on 2026-07-11 with eleven `gh search code` queries (three per framework, two for CrewAI), retaining the top 40 results per query in GitHub’s default relevance order and cloning the first 180 unique repositories in that order; within a cloned repository every `.py` file was extracted (no per-file cap). After removing one repository of our own test fixtures (8 files; a contamination detected during revision and excluded from every number), the corpus is **922 extracted file-level graphs** from 252 distinct repositories (LangGraph 400, AutoGen 359, CrewAI 113, ADK 50), with provenance recorded as repository, commit SHA, and file path. This is a *search-ranked convenience corpus*, not a probability sample: the ranking function is proprietary and non-stationary and the index denominator unpublished, so inclusion probabilities are *undefined* rather than merely unestimated. Every rate is a property of this corpus and the instrument applied to it; we make no claim of the form “ $X\%$ of agent workflows on GitHub.” Mined graphs are small (median 2 non-sentinel nodes, mean 3.9) and predominantly static linear pipelines, and the corpus is *not* mostly applications: 32% of files carry tutorial/test/demo names, 170 graphs are topological duplicates, and a source-level classification of the validation sample found 44% tutorial, 17% toy demo, 7% test, and 33% application-like code.

Table 1: Audit questions and the evidence used to answer each one.

Question	Evidence	Comparison
Extraction effect	matched graphs	mined $\rightarrow$ reconstructed
Policy applicability	source triage	blanket rule $\rightarrow$ HGP-1
Miss mechanism	HGP-1 cases	mined $\rightarrow$ reconstructed

Reference graphs and validation. On a 119-graph validation sample from 87 repositories (LangGraph 40, CrewAI 20, AutoGen 35, ADK 24) we ran two independent LLM-agent passes (Claude Opus 4.8 under a multi-agent harness; all outputs released). First, an agent reconstructed a reference graph from source alone, blind to the extractor’s output and following its node-id conventions; we score node/edge precision–recall and node-kind accuracy against these *LLM-reconstructed reference graphs* (confidence: 77 high, 33 medium, 9 low). Second, every flag was triaged from source (`REAL/EXTRACTION-ARTIFACT/INTENTIONAL`) and an adversarial verifier attempted to overturn each label (4.6% overturned; unresolved labels recorded as `ARGUABLE`). Reference graphs are LLM reconstructions, not human annotations, and both passes share a model family. Results are stable on the high-confidence subset, every triage label carries a source-line justification, and a re-audit overturned one label of 16 re-examined. **Two-annotator independent human validation is pending; the protocol, locked samples, and scoring code are included with the submission.** Wilson intervals in the supplement are binomial descriptive summaries only; repository-clustered bootstraps measure sensitivity to repository composition. Neither has design-based population coverage under the selection procedure below.

How the validation sample was selected. The 119 graphs were chosen under fixed per-framework quotas *without randomization*: the identifiers exist only as literal lists in the released harness scripts, with no seed or reproducible selection rule, so we do not claim the sample is random within framework, nor call it stratified. The framework-share reweighting is consequently a *corpus-share-weighted descriptive statistic*—the rate the corpus would show if each framework’s sampled rate held across its members, correcting the sample’s $\sim 5\times$ over-representation of ADK—not a design-based estimate. We treat it only as a sensitivity analysis and make no inferential claim from its intervals.

A keying defect. We report an instrumentation error found while preparing this comparison, because it changes the numbers. Graphs were keyed by repository plus file *basename*, which is not unique: **76** slugs collide, and for those the two instruments scored *different source files* against the same reference graph. No colliding pair is repairable (supplement), so every colliding graph is dropped: **9** inside the fidelity set, cutting it from 115 matched graphs to the $n=106$ of Table 2, with 4 more of the 119 sampled unavailable at re-mining. The cost is not uniform—ADK loses 5 of 23 versus 0–6% elsewhere—so ADK’s row is the least stable. The keying is fixed in the released instrument.

Table 2: Extractor fidelity on the *matched, provenance-disambiguated* set ($n=106$): both instruments scored on identical graphs, colliding records excluded. Edge P and kind are identical between instruments except where shown—the corrected instrument changes *exactly one* framework.

Framework	n	Edge P	Edge R_{v1}	Edge R_{v2}	Kind
LangGraph	38	0.954	0.678	0.836	0.659/0.670
CrewAI	20	0.700	0.692	0.692	0.929
AutoGen	30	0.966	0.778	0.778	0.890
ADK	18	0.394	0.343	0.343	0.920
Overall	106	0.814	0.652	0.709	0.820/0.824

Extraction fidelity is the bottleneck—for structural checks. Node detection stays strong and stable across instruments ($v2$ overall $P=0.914$, $R=0.868$; $v1$ 0.914/0.862), but edge recall and node-kind accuracy are the bottleneck and are strongly framework-dependent (Table 2): the corrected instrument recovers only 0.34 of ADK’s edges—whose nested `SequentialAgent/ParallelAgent` composition, and AutoGen’s participant lists with dynamic speaker selection, encode topology *implicitly*—versus 0.84 on LangGraph’s explicit edges. On LangGraph the dominant remaining error is instead node-kind (`tool`→`llm`: nodes invoking tools in their body with no declared binding; kind accuracy 0.67), the direct cause of the risk-aware gate’s blindness below. This is a property of the static AST fallback, not of the runtime extractors (for ADK the runtime graph is far richer, 16 vs. ~ 5 edges); realizing that ceiling would require sandboxed execution of untrusted code, which we do not attempt.

A corrected-instrument ablation isolates the effect of extraction changes on the structural checks. The correction bundles *three* behavioral changes—a LangGraph three-argument `add_conditional_edges` fix, a LangGraph declared-tool metadata fix, and ADK `LoopAgent` support—but, re-extracting at identical pinned commits with the check algorithms untouched, only *one* has a measurable effect: LangGraph edge recall rises $0.678 \rightarrow 0.836$, and of the 54 true edges newly recovered, **54/54** are targets of a three-argument `add_conditional_edges` path map across 24 call sites. The other three frameworks are bit-identical between instruments (Table 2), so `LoopAgent` cannot be credited with an improvement. On the matched collision-free corpus ($n=853$) the correction removes roughly a quarter of the structural flags ($299 \rightarrow 229$ flagged workflows; unreachable-exit flags $81 \rightarrow 29$), confirming extraction as the binding constraint *for the structural checks*—a scope the attribution below makes precise, and which does not extend to the policy checks. Both instruments are released.

Flag precision on extracted graphs. What fraction of the checker’s flags is genuine? The as-mined study raised 186 flags on 114 file-level graphs; adversarial triage classified 2 as genuine (both missing human gates), classifying 42% as extraction artifacts and 50% as intentional design—a workflow-level PPV of $2/114 = 1.75\%$ and a flag-level PPV of $2/186 = 1.08\%$. Deduping correlated flags to root cause ($186 \rightarrow 161$) gives 1.24%, and counting all 12 ARGUABLE

labels pessimistically bounds the rate at 7.53%. *None of the 72 structural flags was classified as genuine*, and structural flags are almost entirely a LangGraph phenomenon—partly by construction, since the other three extractors emit connected topologies and cannot produce a dead end or unreachable exit. This as-mined universe is *distinct* from the reconstructed-graph re-run below; the two are never pooled.

Attribution depends on check family. A precision number says findings are non-actionable but not *why*, and the answer decides what to fix. We attribute each of the 186 flags to one of four causes: EXTRACTOR (the graph misrepresents the code); CHECKER (check logic wrong given a correct graph); POLICY-SPEC (graph and check both correct, but the policy does not apply—a blunt human-presence requirement on a read-only tutorial); and LABEL (disputed judgment). Across this selected mixture, extraction artifacts account for 79 flags and policy misspecification for 93. This aggregate is not a universal error budget: it is determined by the check mix, and 113 of 186 findings come from the blanket human-presence rule.

Because the triage vocabulary asked “is this a real defect?”, not “whose fault is this flag?”, we separate CHECKER from POLICY-SPEC *mechanically*, with a survival test: re-run each check on the reference graph. A flag the reconstructed graph removes is EXTRACTOR; a *structural* flag that survives on a faithful graph yet is still not a defect indicts the check’s decision procedure (CHECKER); a surviving *policy* flag indicts its specification. Where the reference graph is itself known wrong—a GT-ERROR label, or a HUMAN node the placebo audit found non-discharging—we record REFERENCE-ERROR rather than force a side. This label-independent instrument agrees with the label-based one on **173 of 186** flags (93.0%) and closes the gap the earlier vocabulary left open: CHECKER accounts for 2 flags, POLICY-SPEC for 94, EXTRACTOR for 71, and REFERENCE-ERROR for 5. Check-side error is almost entirely *specification*, not *logic*.

Controlled substitutions. Attribution rests on labels, so we ran the symmetric ablation: hold the checks fixed, vary only the graph, using the 119 source-reconstructed graphs as a higher-fidelity substitute. Flags fall from 184 to 122, removing **62 flags**. A declaration-sensitive human-gate rule moves the 113 blanket firings to 0 on mined graphs, but the source audit below shows why this is not a successful policy fix: extraction exposes no sensitive bindings. The survival test’s family split is the interpretable comparison. Among the non-actionable *structural* flags, 68 of 72 are extraction-attributable and 2 are checker-attributable. Among the 112 non-actionable *human-gate* flags, 3 are extraction-attributable and 94 are policy-attributable; the remainder are reference errors or disputed labels. Because human-gate checks are 61% of all flags, pooling the families obscures the actionable result: extraction repair targets reachability checks, while policy applicability targets blanket approval checks.

The same test, run on misses. Everything above decomposes false *positives*. Flag triage cannot see misses, so we run the ablation in the other direction over the 12 HGP-1 vio-

Table 3: Distinct quantities from the 119-file source audit. Counts are LLM-assisted pending independent human validation and are not population prevalence estimates.

Quantity	Observed	Interpretation
Flag precision	2/186	actionable findings
Structural defect	1/119	topology judgment
HGP-1	12/9/11	violation/compliant/arguable [†]

[†] Among the 32 files screened as side-effecting; the full audit denominator is 119.

lations from the source audit. The as-mined risk-aware check reports **none** of them, and the cause is unmixed: no mined graph declares a sensitive tool at all, so all 12 misses are extraction-attributable. Replacing the extractor with the reference graph—the same graph substitution—recovers only **1 of 12**. Of the 11 still missed, **9** have the effect absent even from the *reconstruction*, because it lives in a node *body* that the graph abstraction does not represent; 1 is a reconstruction that dropped statically declarable tools; 1 is a workflow the check cleared on a gate that cannot block. The two directions therefore have opposite error budgets: non-actionable findings are check-side, but misses are model-side—and a source-reconstructed graph repairs almost none of them, because the missing information sits *below* the abstraction rather than above it—the same boundary the certification gate runs into (Section 2).

Defining a defect. We re-ran every check on the 119 reference graphs—which can express dead ends and unreachable exits for all four frameworks—and adversarially triaged the results, recovering two genuine defects the extracted study missed. A *structural defect* is a topology error; a *human-gate policy violation* is a normative judgment that a side-effecting tool is reachable with no intervening HUMAN checkpoint, which a permissive policy would not yield. We report each as a **distinct estimand**, their union only as a secondary summary.

Three audit quantities. Table 3 keeps detector precision, structural defects, and a normative policy judgment separate. The one observed structural defect is a router miswiring in an ADK SQL agent. HGP-1 labels 12 of 119 files violations, but its informative denominator is the 32 files screened as containing a source-level side effect: 12 violation, 9 compliant, and 11 arguable. These raw counts, not a reweighted prevalence, are the result. Corpus-share reweighting and repository-cluster sensitivity analyses remain in the supplement.

One policy, applied uniformly. The three policy cases above came from *two* different checks (blunt `human_presence` and path-sensitive `human_gate_coverage`) over two graph populations. That is not one estimand, and it cannot see a violation neither check flags. We therefore fixed one operational policy before case labeling (HGP-1; in the supplement) and applied it to every sampled workflow with a side-effecting tool at *source* level—all 32—regardless of flag universe.

HGP-1 fixes the side-effecting categories by what the callee does in source rather than by name; requires a gate to be an explicit checkpoint or a blocking confirmation whose negative branch skips the effect, and to *dominate* the call site; and records undecidable cases as ARGUABLE, never silently as compliant. Demo, tutorial, and test status are *not* exemptions; genuine mocks and visibly sandboxed effects are.

The audit returns **12 violations, 9 compliant, 11 arguable**, replacing the ad-hoc two-check count of 3/119. Of those three earlier cases, HGP-1 retains one and moves two to ARGUABLE; it also finds 11 cases the earlier procedure missed. Only the 32 effect-bearing files were read *under HGP-1*; the other 87 enter the denominator as non-violations by construction, having been excluded by the same source-level classification pass that produced the 32. An independent sweep of all 87 sources for effect signatures flagged 10 for manual re-reading: 9 are correctly screened and 1 is borderline—an MCP workbench whose server module lies outside the mining unit—which HGP-1 would record as ARGUABLE, not a violation. This check found no missed violation, but does not prove the screen complete. Three of the 12 lie *outside both flag universes*—no triage entry, neither check fired—confirming that false negatives exist beyond the union of the flag sets, where a flag-triage study can never find them.

Placebo gates: the reference graphs overstate oversight.

The most consequential finding is not a count. All seven side-effecting workflows that *both* checks cleared were cleared because the reference graph held a node typed HUMAN—and on source inspection *none is a genuine dominating gate*: five are placebo or inert, one does not dominate the call site, one is unverifiable. One exemplar declares six `*_human_approval` nodes whose body reads a constructor flag hard-set to `True`: a checkpoint that cannot block, typed HUMAN by both extractor and reconstruction because it is named and shaped like one. Reference graphs assign HUMAN from naming and framework type, not from whether the body can block, so *every* human-gate estimand computed over them—including the risk-aware one—is biased downward on these observed cases. The fidelity metric compounds it: for two of these workflows it records human-node recall 1.0, crediting the extractor with recovering oversight that does not exist. A source-reconstructed graph is not a semantic oracle.

Risk-aware gating. The blunt human-presence check fires on 91% of mined graphs with only 2 of 113 firings genuine—uninformative as a detector, motivating the risk-aware *human-gate coverage* check: flag only when a sensitive tool (destructive/write/exec/communication/financial, by a keyword lexicon over bound tool names) is reachable without a HUMAN gate. On the 922 mined graphs it fires zero times—a statement about extracted *declarations*, not the code: no mined graph declares a sensitive binding, while source review finds side-effecting tools in 32 of 119 sampled workflows, invisible through the `tool→llm` kind error. On reconstructed graphs it fires three times. Its recall is bounded by extraction fidelity, its precision—per the audit above—by whether a HUMAN node can block.

Threats to validity. *Construct:* the composite unions a structural property with a policy judgment, so we report the two separately. *Internal and label reliability:* reference graphs and triage labels come from LLM passes sharing a model family, so correlated errors are possible; confidence stratification, adversarial overturn rates, and a re-audit do not establish correctness. The included two-annotator protocol is pending and is the principal remaining validity gap. *External:* the frame is public GitHub files and the unit a file-level graph, so results may not transfer to executable roots or private code. *Sampling:* the frame is a convenience corpus with undefined inclusion probabilities, quota-selected without randomization. *Reference-graph semantics:* node kinds are assigned from naming, not from whether a body has the effect its kind implies—the placebo-gate audit shows this can bias human-gate judgments downward. All rates are descriptive audit proportions, not population estimates.

4 Controlled Evaluation and Monitor Selection

Controlled detection test. A curated corpus of 18 workflows across all four frameworks, seeded with known anti-patterns, confirms the checks fire when defects are present: the analyzer detects every injected structural defect and, with a risk-aware sensitive-tool set, flags 8 workflows whose sensitive path is reachable without a human gate, versus 10 for the blunt check—declining to flag two that lack a human node but perform no sensitive action. This is a controlled test that the algorithms work, not evidence of prevalence. Verification is sub-second on synthetic graphs of 5,000 nodes; the supplement compares Promela/SMV modeling effort.

Static monitor selection. The graph $\times$ DFA product gives the static machinery a job independent of whether defects exist: deciding which runtime monitors to deploy. Its value is *soundness*, not scale, and it rests on the certification gate of Section 2, so every pruning number is sound *relative to an authored abstraction*—we assume, and do not prove, that a curated workflow’s nodes emit no events outside their declared bindings. A policy whose product is safe can never fire at runtime, so its monitor need not be deployed; `may_violate` is never suppressed. On the curated corpus the gate classifies 237 of 270 monitor instances (87.8%) as inert under that assertion and declines 14 (all on one seeded-livelock workflow) as *inconclusive*, leaving 19 to runtime enforcement.

Descriptive vs. sound pruning on mined code. This rate does not survive lossy extraction. A $\sim 93\%$ “pruning” rate is available on the mined graphs, but it is pure *alphabet-inertness*: all 13,830 mined (workflow, policy) instances are inert only because no mined workflow invokes any of the 15 policies’ tools—a mismatched vocabulary, not a sound prune. On a may-provenance graph a missing edge could hide exactly the tool that makes a live monitor look inert, so the gate certifies *none* of them, refusing all as uncertified extraction.

Does path-sensitivity buy anything? Alphabet-inertness needs no product; the product earns its keep only when a policy’s atoms occur and inertness turns on *which* paths reach

them. On 18 (workflow, policy) pairs of that kind, a runtime-only strategy certifies 0, since one unobserved path forbids the claim; alphabet-only prunes 1, node-reachability 3, and the path-sensitive product 10—with no false prunes across 50 concrete executions. We are deliberately conservative: five of the 9 incremental cases are linear response-obligation pipelines in which the consequent trivially dominates every path, and the branching and looping evidence rests on 4 purpose-built *synthetic* fixtures. Fifty executions cannot establish the absence of false prunes; this is a mechanism demonstration, not evidence of scale.

5 Related Work

Runtime agent safety. NeMo Guardrails (NVIDIA 2023), Guardrails AI (Guardrails AI 2023), LlamaGuard (Inan et al. 2023), and Rebuff (Protectai 2023) enforce safety at the point of LLM output or tool call, catching violations only when the offending path runs; ToolEmu (Ruan et al. 2024) and AgentDojo (Debenedetti et al. 2024) measure agent risk by executing behaviors. Our analyzer is complementary: it analyzes topology before deployment and decides which runtime monitors are needed (Section 4).

Concurrent agent-policy systems. A concurrent line of work enforces policies at the tool-call boundary with formal machinery: Agent-C compiles temporal safety specifications to constraints checked at generation time with conformance guarantees (Kamath et al. 2026); AgentSpec provides a runtime-enforcement DSL (Wang, Poskitt, and Sun 2025); ClawGuard and SecureClaw interpose rule sets and authorization boundaries against prompt injection (Zhao et al. 2026; Ma and Schmid 2026); PolicyGuard trains a guardrail model for trajectory-level violation detection (Wen et al. 2025); and AgentFlow builds cross-framework agent dependency graphs for static security analysis at scale (Wang et al. 2026). Two bound our contribution sharply. Agent-C already enforces temporal specifications with conformance guarantees, so we claim no novelty for temporal enforcement; AgentFlow already builds richer cross-framework dependency graphs over 5,399 programs, so we claim none for cross-framework graph construction or scale. What remains ours is the measurement neither performs: *how faithfully* such extracted models correspond to the code they abstract. AgentFlow reports no validation of its graphs against reference models; our result is that on public code this is not a formality but a dominant term in the error budget.

Empirical characterizations of agent defects. Recent studies catalogue where agent systems break: a defect taxonomy from developer discussions (Ning et al. 2024), nine root causes across $\sim 1,026$ orchestration bugs (Xue et al. 2025), and 221 vulnerabilities spanning fourteen types (Shen et al. 2025). We ask not *which* defects occur, but *how reliably a static analyzer can recover the behavioral model it needs to find them*.

Model checking and workflow soundness. Classical model checkers—SPIN (Holzmann 1997), NuSMV (Cimatti et al. 2002), CBMC (Kroening and Tautschnig 2014)—verify the same class of properties but require manual translation

into Promela/SMV/annotated C. Our policy semantics follow LTL on finite traces (De Giacomo and Vardi 2013), with monitoring in the style of JavaMOP (Jin et al. 2012) and three-valued runtime verification (Bauer, Leucker, and Schallhart 2011). Business-process soundness for Petri nets (van der Aalst 2011) and BPMN (Dijkman, Dumas, and Ouyang 2008) studies analogous reachability properties over visual notations rather than the programmatic APIs agent frameworks expose.

Audits of static analysis. Industrial experience established that false-positive and actionability rates, not detector cleverness, decide adoption (Bessey et al. 2010; Johnson et al. 2013; Sadowski et al. 2018). Ours is the agent-workflow instance of that genre, and adds a failure mode those studies did not face: the *model extraction* step carries its own large share of the error budget, so non-actionability must be apportioned between extractor and policy rather than attributed wholly to the checks. Broader agent-safety work emphasizes alignment, capability evaluation, and governance (Anthropic 2023; METR 2024; Gabriel et al. 2024); our contribution is orthogonal and empirical. The reported rates describe a fixed convenience-corpus audit, not population prevalence.

6 Conclusion

Static checks on agent workflows are only as credible as the behavioral model they run on and the applicability of the policy they enforce. In our audit of 922 extracted file-level graphs from 252 public GitHub repositories, the causes are check-family dependent: graph substitution attributes 68 of 72 non-actionable structural findings to extraction, while it attributes 94 of 112 non-actionable human-gate findings to an inapplicable approval policy. Pooling them into one error rate would make the result depend on how many checks of each family happened to run.

The reverse audit is more consequential. All 12 source-audited HGP-1 violations escape the mined pipeline, and a source-reconstructed graph recovers only one; nine regulated effects live inside node bodies outside the graph vocabulary. Every side-effecting workflow cleared by both graph checks was cleared by a *placebo* gate. Thus graph reconstruction, policy applicability, and semantic abstraction coverage are separate validation obligations. The static machinery retains a limited operational role: on authored graphs whose event coverage the caller asserts, the graph $\times$ DFA product can select runtime monitors; no mined graph in this study receives a *safe* verdict.

Limitations. The frame is public GitHub, not production systems, and two thirds of the validation sample is tutorial/demo/test code; the unit is an extracted file-level graph, not an executable workflow root. The 119 files were quota-selected without randomization, so all proportions describe this audit rather than a population. Reference graphs and labels remain LLM-assisted pending the included two-annotator validation. The result redirects effort toward higher-fidelity extraction, body-level effect modeling, and per-workflow policy instantiation.

Artifact. An anonymized artifact is submitted, not promised: the tool, both corpora, the mining and validation pipeline, reference graphs, justified triage labels, and all evaluation scripts.

Ethical Statement

We mine only *public* GitHub repositories, record exact provenance (repository at commit SHA plus file path), and re-distribute only derived graph abstractions, never third-party source; no personal data is processed. Findings classified as defects are disclosed to the affected repositories before publication, without naming maintainers. Generative AI tools assisted with language editing, software and claim review, source-triage labels, and submission-compliance auditing. The authors verified all text, citations, code, and results, take responsibility for the complete submission, and list no AI system as an author or scientific source.

References

- Anthropic. 2023. Anthropic’s Responsible Scaling Policy. <https://www.anthropic.com/responsible-scaling-policy>. Accessed: 2026-03-01.
- Bauer, A.; Leucker, M.; and Schallhart, C. 2011. Runtime Verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4).
- Bessey, A.; Block, K.; Chelf, B.; Chou, A.; Fulton, B.; Hallem, S.; Henri-Gros, C.; Kamsky, A.; McPeak, S.; and Engler, D. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Communications of the ACM*, 53(2): 66–75.
- Cimatti, A.; Clarke, E.; Giunchiglia, E.; et al. 2002. NuSMV 2: An OpenSource Tool for Symbolic Model Checking. In *International Conference on Computer Aided Verification (CAV)*. Springer.
- Clarke, E. M.; Grumberg, O.; and Peled, D. A. 1999. *Model Checking*. MIT Press.
- CrewAI. 2025. CrewAI documentation. <https://docs.crewai.com>. Accessed: 2026-02-28.
- De Giacomo, G.; and Vardi, M. Y. 2013. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In *International Joint Conference on Artificial Intelligence (IJCAI)*.
- Debenedetti, E.; Zhang, J.; Balunović, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.
- Dijkman, R. M.; Dumas, M.; and Ouyang, C. 2008. Semantics and analysis of business process models in BPMN. *Information and Software Technology*, 50(12): 1281–1294.
- Gabriel, I.; et al. 2024. The Ethics of Advanced AI Assistants. *arXiv preprint arXiv:2404.16244*.
- Google. 2025. Google Agent Development Kit (ADK) documentation. <https://google.github.io/adk-docs/>. Accessed: 2026-02-28.

- Guardrails AI. 2023. Guardrails AI: Adding Guardrails to Large Language Models. <https://github.com/guardrails-ai/guardrails>. Accessed: 2026-03-01.
- Holzmann, G. J. 1997. The Model Checker SPIN. *IEEE Transactions on Software Engineering*, 23(5): 279–295.
- Inan, H.; Upasani, K.; Chi, J.; et al. 2023. LlamaGuard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674*.
- Jin, D.; Meredith, P. O.; Lee, C.; and Roşu, G. 2012. JavaMOP: Efficient Parametric Runtime Monitoring Framework. In *International Conference on Software Engineering (ICSE)*.
- Johnson, B.; Song, Y.; Murphy-Hill, E.; and Bowdidge, R. 2013. Why Don't Software Developers Use Static Analysis Tools to Find Bugs? In *International Conference on Software Engineering (ICSE)*.
- Kamath, A.; Zhang, S.; Xu, C.; Ugare, S.; Singh, G.; and Misailovic, S. 2026. Enforcing Temporal Constraints for LLM Agents. Preprint. <https://github.com/structuredllm/agent-c>.
- Kroening, D.; and Tautschnig, M. 2014. CBMC – C Bounded Model Checker. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer.
- LangChain. 2025. LangGraph documentation. <https://docs.langchain.com/oss/python/langgraph/overview>. Accessed: 2026-02-28.
- Ma, Y.; and Schmid, S. 2026. SecureClaw: Clawing Back Control of LLM Agents. *arXiv:2606.09549*.
- METR. 2024. METR: Model Evaluation and Threat Research. <https://metr.org>. Accessed: 2026-03-01.
- Ning, K.; Chen, J.; Zhang, J.; Li, W.; Wang, Z.; Feng, Y.; Zhang, W.; and Zheng, Z. 2024. Defining and Detecting the Defects of the Large Language Model-based Autonomous Agents. *arXiv preprint arXiv:2412.18371*.
- NVIDIA. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. <https://github.com/NVIDIA/NeMo-Guardrails>. Accessed: 2026-03-01.
- Protectai. 2023. Rebuff: Self-hardening Prompt Injection Detector. <https://github.com/protectai/rebuff>. Accessed: 2026-03-01.
- Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations (ICLR)*.
- Sadowski, C.; Aftandilian, E.; Eagle, A.; Miller-Cushon, L.; and Jaspan, C. 2018. Lessons from Building Static Analysis Tools at Google. *Communications of the ACM*, 61(4): 58–66.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.
- Shen, Z.; Dai, J.; Zhang, Y.; and Yang, M. 2025. Security Debt in LLM Agent Applications: A Measurement Study of Vulnerabilities and Mitigation Trade-offs. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- van der Aalst, W. M. P. 2011. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer.
- Wang, H.; Poskitt, C. M.; and Sun, J. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. *arXiv preprint arXiv:2503.18666*.
- Wang, S.; Hou, X.; Zhao, Y.; Cheng, X.; and Wang, H. 2026. AgentFlow: Building Agent Dependency Graphs for Static Analysis of Agent Programs. *arXiv:2607.01640*.
- Wen, X.; Mo, W. J.; Xie, Y.; Qi, P.; and Chen, M. 2025. Towards Policy-Compliant Agents: Learning Efficient Guardrails for Policy Violation Detection. *arXiv:2510.03485*.
- Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling (COLM)*.
- Xue, Z.; Zhao, Y.; Wang, S.; Chen, K.; and Wang, H. 2025. A Characterization Study of Bugs in LLM Agent Workflow Orchestration Frameworks. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
- Zhao, W.; Li, Z.; Zhang, P.; and Sun, J. 2026. ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection. *arXiv:2604.11790*.